DATA STRUCTURES & ALGORITHMS · C++

Recursion & Backtracking in C++ From the Call Stack to Placement-Ready

Everything for recursion and backtracking: the call stack, the leap of faith, how to analyse a recursive function's cost, divide and conquer, the one backtracking template, subsets, combinations, permutations, N-Queens, Sudoku, partitioning, the bridge to DP, the 12 patterns, and a curated problem list.

61	12	37	38
SECTIONS	PATTERNS	CODE BLOCKS	PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 14 October 2026

Contents

PART 0 — Recursion, from absolute zero

0.1 The picture — what the machine actually does

0.2 Before versus after the call

0.3 The vocabulary

0.4 Stack overflow, and how deep is too deep

0.5 How to recognise the topic in a problem statement

PART 1 — The recursion mental model

1.1 The leap of faith

1.2 The three questions — answer them before writing code

1.3 Three ways to get information in and out

1.4 Pass by value versus pass by reference

1.5 The recursion tree

PART 2 — Tracing, and the call stack in detail

2.1 The table method

2.2 The classic "what does this print?" questions

2.3 Every recursion can be written with an explicit stack

2.4 Tail recursion

PART 3 — The cost of a recursive function

3.1 Time = nodes in the recursion tree × work per node

3.2 Space = maximum depth × space per frame

3.3 The recurrences you must recognise on sight

3.4 Why fast power needs the variable half

PART 4 — Warm-ups, and divide and conquer

4.1 The reduce-by-one family

4.2 Tower of Hanoi — the purest leap of faith

4.3 Recurrences you write directly from the definition

4.4 Recursion on linked lists and strings

4.5 Recursion on a stack, with no other container

4.6 Divide and conquer

PART 5 — Backtracking: the one template

5.1 The idea

5.2 The template

5.3 The undo must exactly mirror the do

5.4 Two shapes of the same tree

5.5 Pruning — the difference between TLE and accepted

PART 6 — Subsets and subsequences

6.1 All subsets (LC 78) — pick / skip

6.2 Subsets with duplicates (LC 90) — sort, then skip at the same depth

6.3 Subsets with a bitmask

6.4 Subsequences with a property — return a value instead of building lists

6.5 When subsets turn into DP

PART 7 — Combinations and combination sums

7.1 Choose k from n (LC 77)

7.2 The combination-sum family

7.3 Letter combinations of a phone number (LC 17)

PART 8 — Permutations

8.1 Distinct values (LC 46) — the used array

8.2 The swap method

8.3 Permutations with duplicates (LC 47)

8.4 The k-th permutation directly (LC 60)

8.5 Next permutation

PART 9 — Constraint backtracking on boards and grids

9.1 N-Queens (LC 51, LC 52)

9.2 Sudoku solver (LC 37)

9.3 Word search (LC 79) — mark, explore, unmark

9.4 The rest of the grid family

PART 10 — Partitioning and constrained generation

10.1 Cut at every position

10.2 Generation with counters (LC 22)

10.3 Remove Invalid Parentheses (LC 301)

10.4 Expression Add Operators (LC 282) — the last trick

PART 11 — From recursion to memoization: the bridge to DP

11.1 Overlapping subproblems

11.2 When does memoization help?

11.3 The recipe

PART 12 — The complete pattern taxonomy

How to tell 5 and 6 apart

How to tell 6 and 7 apart

PART 13 — How to approach ANY recursion or backtracking problem

Step 1 — Read the constraints first

Step 2 — Decide: enumerate, count, or search?

Step 3 — Draw the top two levels of the tree

Step 4 — Fill the five blanks

Step 5 — Add the pruning

Step 6 — Dry-run the smallest inputs

PART 14 — The bug catalogue

PART 15 — Complexity cheat sheet

PART 16 — The problem list

PART 17 — A three-week plan

Quick reference card

PART 0 — Recursion, from absolute zero

A recursive function is a function that **calls itself on a smaller version of the same problem**. That is the entire definition. Everything difficult about recursion comes from one fact: you have to believe the smaller call works *before you have finished writing it*.

You have already done this. In Trees you wrote:

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

and in Stacks & Queues you saw that every one of those calls is a frame pushed onto the **call stack**. This topic takes that idea and makes it the main tool, not a side effect.

Every recursive function has exactly two parts:

- A **base case** — the smallest input, answered directly without recursion. It is what stops the recursion.
- A **recursive case** — break the input into something smaller, call yourself on it, and use the answer.

```
int factorial(int n) {
    if (n == 0) return 1;              // base case: 0! = 1
    return n * factorial(n - 1);       // recursive case: n! = n * (n-1)!
}
```

> **The single sentence to remember:** a recursive function solves *one step* of the problem and hands the rest to a smaller copy of itself. You write the step; you **trust** the copy.

And **backtracking** is recursion used to *explore choices*: at each step you try every option, recurse to see where it leads, and then **undo** the option before trying the next one. Subsets, permutations, N-Queens, Sudoku and word search are all that one idea.

0.1 The picture — what the machine actually does

Every call gets its own **stack frame**: its own copy of the parameters and local variables, plus a note of where to return to. Calls push frames; returns pop them.

```
factorial(3)                          the call stack, at its deepest

factorial(3)  waits for f(2)
  factorial(2)  waits for f(1)          ┌──────────────────┐
    factorial(1)  waits for f(0)        │ factorial(0)  n=0 │ <- top: returns 1
      factorial(0)  returns 1           ├──────────────────┤
    returns 1 * 1 = 1                   │ factorial(1)  n=1 │
  returns 2 * 1 = 2                     ├──────────────────┤
returns 3 * 2 = 6                       │ factorial(2)  n=2 │
                                        ├──────────────────┤
                                        │ factorial(3)  n=3 │
                                        ├──────────────────┤
                                        │ main()            │
                                        └──────────────────┘
```

Two phases, and you must be able to see both:

- **Going down (the "winding" phase)** — each call is suspended at the line `n * factorial(n - 1)`, waiting.
- **Coming back up (the "unwinding" phase)** — each frame receives the answer from below, finishes its multiplication, and returns.

Work you write **before** the recursive call happens on the way down. Work you write **after** it happens on the way back up. This one observation explains most recursion puzzles in written rounds.

0.2 Before versus after the call

```
void f(int n) {
    if (n == 0) return;
    cout << n << " ";      // BEFORE the call: runs on the way down
    f(n - 1);
}
// f(3) prints: 3 2 1

void g(int n) {
    if (n == 0) return;
    g(n - 1);
    cout << n << " ";      // AFTER the call: runs on the way back up
}
// g(3) prints: 1 2 3
```

Same calls, same order of calls, opposite output. `g` is how you print a linked list in reverse without reversing it, and how postorder traversal works.

0.3 The vocabulary

TERM	MEANING
Base case	The input answered directly; it stops the recursion
Recursive case	The step that calls the function on a smaller input
Stack frame	One call's private parameters and locals, held on the call stack
Recursion depth	The largest number of frames alive at once — this is the **space** cost
Recursion tree	A drawing with one node per call and an edge to each call it makes — this gives the **time** cost
Head recursion	The recursive call comes first, the work after it (`g` above)
Tail recursion	The recursive call is the very last thing the function does (`f` above)
Linear recursion	Each call makes at most **one** recursive call (factorial)
Tree recursion	A call makes **two or more** recursive calls (Fibonacci, subsets)
Backtracking	Try a choice, recurse, then **undo** the choice and try the next
State space tree	The recursion tree of a backtracking search; each root-to-leaf path is one sequence of choices
Pruning	Returning early from a branch that cannot lead to an answer
Memoization	Caching each subproblem's answer so it is computed once (the bridge to DP)

0.4 Stack overflow, and how deep is too deep

Each frame costs memory — typically a few dozen to a few hundred bytes. The default stack is around **1 MB on Windows** and **8 MB on Linux**. That means:

- Depth of a few **thousand**: always fine.
- Depth around **10^5**: usually fine on Linux judges (LeetCode, Codeforces), risky on Windows and with big frames.
- Depth around **10^6** or more: expect a crash. That is a **stack overflow** — a segmentation fault, not an exception you can catch.

Missing base cases give **infinite** depth and always overflow. When your program dies with `Segmentation fault` or exit code `0xC00000FD` on Windows and you used recursion, check the base case first.

> **The rule for interviews:** if the recursion depth is O(n) and n can be 10^5 or more (a skewed tree, a linked list, a long string), say out loud that the iterative version avoids stack overflow. Interviewers listen for that sentence.

0.5 How to recognise the topic in a problem statement

THE PROBLEM SAYS	REACH FOR
"all subsets", "power set", "all subsequences"	pick / skip recursion
"all combinations", "choose k", "sum to target"	backtracking with a `start` index
"all permutations", "all arrangements", "orderings"	backtracking with a `used` array
"generate all valid …" (parentheses, IPs, expressions)	backtracking with validity rules
"partition the string so that every piece …"	cut-at-every-position backtracking
place N things with no conflicts — queens, Sudoku, colouring	constraint backtracking
find a word / path in a grid, each cell used once	grid DFS with mark and unmark
"count the number of ways" with small n (≤ 20)	recursion; add memo if subproblems repeat
n ≤ 10–20 in the constraints	exponential is *intended* — backtracking
"divide into halves", merge sort, "count inversions"	divide and conquer
a definition that refers to itself (grammar rows, Josephus, Hanoi)	write the recurrence directly

The constraints are the strongest signal of all. **n ≤ 20** means $2^n \approx 10^6$ is affordable. **n ≤ 10** means $n! \approx 3.6 \times 10^6$ is affordable. When you see those numbers, the problem is asking you to enumerate.

PART 1 — The recursion mental model

1.1 The leap of faith

The most common way beginners get stuck is trying to trace every call in their head. For `maxDepth` on a 1000-node tree that is impossible, and it is also unnecessary.

The working method is called the **recursive leap of faith**:

1. **Define precisely what the function returns** for any valid input. Write it as a comment.
2. **Assume** that definition already holds for every smaller input.
3. Using that assumption, **write the answer for the current input**.
4. Check the base case.

That is mathematical induction, written as code. If the base case is right and each step is right *given the smaller answers*, the whole thing is right — you never need to trace it.

```
// returns the sum of the digits of n  (step 1: the definition)
int sumDigits(int n) {
    if (n < 10) return n;                 // step 4: base case
    return n % 10 + sumDigits(n / 10);    // step 2+3: trust sumDigits(n/10)
}
```

1.2 The three questions — answer them before writing code

For every recursive function, answer these out loud:

1. **What is the smallest input, and what is its answer?** → the base case.
2. **If a smaller call already worked, how do I use its answer?** → the recursive case.
3. **Does every call move strictly toward the base case?** → termination.

Question 3 is the one people skip. `f(n)` calling `f(n)` (forgot the `-1`), or `f(n/2)` when `n` can be negative, or a binary search with `mid` instead of `mid+1` — all of them violate it and all of them overflow the stack.

1.3 Three ways to get information in and out

A recursive function can move information in three ways, and choosing the right one is half the design:

MECHANISM	DIRECTION	USE IT FOR	EXAMPLE
Parameters	top → down	state accumulated *so far*	`sum` so far, current `path`, index `i`
Return value	bottom → up	an answer computed *from below*	depth of a subtree, number of ways
A member / global / reference	anywhere	collecting results, a running best	`vector<vector<int>> res`, `best`

- **Counting and optimisation** problems usually **return** a value: `return countLeft + countRight;`.

- **Enumeration** problems usually push into a shared `res` and return `void`.
- **Search** problems ("does one exist?") return `bool` and stop as soon as one branch says `true`.

1.4 Pass by value versus pass by reference

This choice decides whether you need to undo anything.

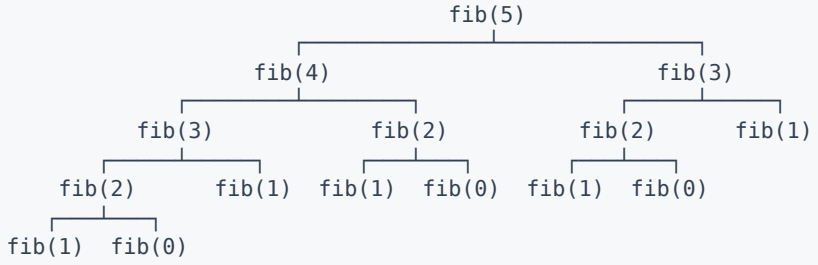
```
// BY VALUE: every call gets its own copy, nothing to undo
void dfs(int i, vector<int> path) {     // copies the vector: O(n) per call
    ...
    path.push_back(x);
    dfs(i + 1, path);                    // the caller's path is untouched
}

// BY REFERENCE: one shared vector, so you MUST undo
void dfs(int i, vector<int>& path) {    // no copy
    ...
    path.push_back(x);
    dfs(i + 1, path);
    path.pop_back();                     // restore it for the next branch
}
```

Integers and small values: pass by value, never undo (`dfs(i + 1, sum + x)` — the caller's `sum` is unchanged). Vectors, strings, boards, `used` arrays: pass by reference and undo. That is the **backtracking discipline**, and Part 5 builds the whole topic on it.

1.5 The recursion tree

Draw one node per call, with an edge to each call it makes. For Fibonacci:



The tree tells you three things at once:

- **Time** ≈ number of nodes × work per node. Here the tree has about 2^n nodes → $O(2^n)$.
- **Space** = the height of the tree, because only one root-to-leaf path is on the stack at a time → O(n).
- **Repetition** — `fib(3)` appears twice, `fib(2)` three times. Repeated subtrees mean memoization will help (Part 11).

When you are stuck on a recursive problem, drawing the top three levels of this tree is almost always the fastest way forward.

PART 2 — Tracing, and the call stack in detail

You will not trace recursion when *writing* it — that is what the leap of faith is for. But written rounds love "what does this print?", and you debug by tracing. So learn to do it mechanically.

2.1 The table method

Write one row per frame, with its parameters, and fill in the return value on the way back up.

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 2) + 1;     // counts the calls in fib's tree
}
```

```
 call      waits on            returns
 f(4)      f(3) + f(2) + 1     5 + 3 + 1 = 9
 f(3)      f(2) + f(1) + 1     3 + 1 + 1 = 5
 f(2)      f(1) + f(0) + 1     1 + 1 + 1 = 3
 f(1)      base case           1
 f(0)      base case           1
```

Fill the bottom rows first, then work upward. That is the unwinding phase made explicit.

2.2 The classic "what does this print?" questions

```
void fun(int n) {
    if (n == 0) return;
    cout << n << " ";
    fun(n - 1);
    cout << n << " ";
}
// fun(3) -> 3 2 1 1 2 3
```

The first `cout` runs on the way down, the second on the way back up. Any function with work on both sides of the call prints a "mountain".

```
void fun2(int n) {
    if (n == 0) return;
    fun2(n - 1);
    cout << n << " ";
    fun2(n - 1);
}
// fun2(3) -> 1 2 1 3 1 2 1     (this is exactly the order Tower of Hanoi moves discs)
```

Two calls per frame: the output is an **inorder traversal** of the recursion tree. $2^n - 1$ lines of output.

2.3 Every recursion can be written with an explicit stack

The call stack is just a stack. Anything recursive can be rewritten iteratively by pushing what the recursive call would have received. You did this for iterative tree traversal in Topic 06 and for "recursion ↔ explicit stack" in Topic 04 Part 2.4.

```cpp
// recursive subsets, written with an explicit stack of (index, path)
vector<vector<int>> subsetsIter(const vector<int>& a) {
    vector<vector<int>> res;
    stack<pair<int, vector<int>>> st;
    st.push({0, {}});
    while (!st.empty()) {
        auto [i, path] = st.top(); st.pop();
        if (i == (int)a.size()) { res.push_back(path); continue; }
        st.push({i + 1, path});                    // skip
        path.push_back(a[i]);
        st.push({i + 1, path});                    // pick
    }
    return res;
}
```

You rarely *need* this for backtracking — depths are small (n ≤ 20) — but it is the answer to "can you do it without recursion?" and it is how you avoid stack overflow when depth is O(n) with large n.

2.4 Tail recursion

A call is a **tail call** when nothing is left to do after it returns:

```cpp
int factTail(int n, int acc = 1) {
    if (n == 0) return acc;
    return factTail(n - 1, acc * n);    // nothing happens after this call
}
```

With optimisation on (`-O2`), GCC and Clang usually turn tail calls into a loop, so depth stays O(1). But **the C++ standard does not guarantee it**, MSVC in debug builds will not do it, and `n * factorial(n - 1)` is *not* a tail call (the multiplication happens after). Know the term; do not rely on the optimisation.

PART 3 — The cost of a recursive function

3.1 Time = nodes in the recursion tree × work per node

That is the whole method. Count how many calls are made in total, multiply by the non-recursive work each one does.

SHAPE OF THE RECURSION	CALLS	EXAMPLE	TIME
one call, n − 1	n	factorial, sum of digits, reverse	O(n)
one call, n / 2	log n	fast power, binary search	O(log n)
two calls, n / 2, plus O(n) merge	n per level × log n levels	merge sort	O(n log n)
two calls, n − 1 and n − 2	$\approx 1.618^n$	naive Fibonacci	$O(2^n)$ loose bound
two calls, n − 1 (pick / skip)	2^n leaves	subsets	$O(2^n \cdot n)$ with copying
n, n − 1, n − 2 … choices	n! leaves	permutations	$O(n! \cdot n)$
k choices per level, depth d	k^d	letter combinations (k ≤ 4)	$O(4^n \cdot n)$

The extra **· n** on subsets and permutations is the cost of *copying* each answer into `res`. Interviewers accept either form; say which one you mean.

3.2 Space = maximum depth × space per frame

Only one root-to-leaf path is alive at any moment. So:

- Recursive space is the **depth**, not the number of calls. Naive Fibonacci makes 2^n calls but uses O(n) stack.
- Do not forget the *output*. Subsets returns 2^n lists — $O(2^n \cdot n)$ memory — but that is usually reported separately as "output size".
- Passing a `vector` **by value** multiplies the frame size by n. That is a hidden $O(n^2)$ of space and time; pass by reference and undo.

3.3 The recurrences you must recognise on sight

RECURRENCE	SOLUTION	WHERE YOU MEET IT
$T(n) = T(n-1) + O(1)$	$O(n)$	factorial, linked-list recursion
$T(n) = T(n-1) + O(n)$	$O(n^2)$	selection sort written recursively, naive "remove one and recurse"
$T(n) = T(n/2) + O(1)$	$O(\log n)$	binary search, fast power
$T(n) = 2T(n/2) + O(1)$	$O(n)$	tree traversal, max of an array by halves
$T(n) = 2T(n/2) + O(n)$	**$O(n \log n)$**	merge sort, reverse pairs, quicksort average
$T(n) = T(n-1) + T(n-2)$	$O(\varphi^n) \approx O(1.618^n)$	naive Fibonacci
$T(n) = 2T(n-1) + O(1)$	$O(2^n)$	Tower of Hanoi, pick / skip
$T(n) = n \cdot T(n-1)$	$O(n!)$	permutations

The Master Theorem covers the `aT(n/b) + f(n)` rows in general. At interview depth you only need the merge-sort row, and the reasoning behind it: **log n levels, O(n) work across each level.**

```
merge sort's recursion tree:  every level does n work in total

level 0            [ n ]                          n
level 1        [n/2]   [n/2]                       n
level 2     [n/4][n/4][n/4][n/4]                   n
 ...                 ...                          ...
level log n   [1][1][1] ... [1]                    n
                                          total: n log n
```

3.4 Why fast power needs the variable `half`

```cpp
// O(log n): one call, reused
double half = power(x, n / 2);
return half * half;

// O(n) — and slower than the plain loop
return power(x, n / 2) * power(x, n / 2);
```

The second version makes **two** calls on n/2, giving $T(n) = 2T(n/2) + O(1) = \mathbf{O(n)}$. It is correct and it is a classic interview trap: the output is right and the complexity is wrong.

PART 4 — Warm-ups, and divide and conquer

These are the recursion problems Indian written rounds ask directly, and they train the leap of faith on inputs small enough to verify by hand. Write each one, then *stop tracing it* and read it as a definition.

4.1 The reduce-by-one family

Each of these makes **one** recursive call on an input one smaller.

```cpp
// sum of 1..n
int sumN(int n) { return n == 0 ? 0 : n + sumN(n - 1); }

// count digits
int countDigits(int n) { return n < 10 ? 1 : 1 + countDigits(n / 10); }

// reverse an array in place, from both ends
void rev(vector<int>& a, int l, int r) {
    if (l >= r) return;
    swap(a[l], a[r]);
    rev(a, l + 1, r - 1);
}

// is the array sorted?
bool isSorted(const vector<int>& a, int i) {
    if (i + 1 >= (int)a.size()) return true;          // 0 or 1 element left
    return a[i] <= a[i + 1] && isSorted(a, i + 1);
}

// gcd, Euclid's algorithm: gcd(a, b) = gcd(b, a % b)
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

// recursive binary search on a sorted array
int bs(const vector<int>& a, int lo, int hi, int x) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    return a[mid] < x ? bs(a, mid + 1, hi, x) : bs(a, lo, mid - 1, x);
}
```

Each has the same shape: **handle one element, trust the rest.** Binary search is the halving version of the same thing.

4.2 Tower of Hanoi — the purest leap of faith

Move n discs from rod A to rod C, one at a time, never putting a larger disc on a smaller one.

Do **not** try to trace it. Read it as a definition:

> To move n discs from A to C: move the top n − 1 out of the way to B, move the biggest disc to C, move the n − 1 from B onto C.

```cpp
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);      // park n-1 on the spare rod
    cout << from << " -> " << to << "\n";   // move the biggest disc
    hanoi(n - 1, via, to, from);      // put the n-1 back on top
}
```

The number of moves satisfies $M(n) = 2M(n-1) + 1$, so **$M(n) = 2^n - 1$**. That formula is the written-round question; know it and know why.

4.3 Recurrences you write directly from the definition

Some problems define something in terms of a smaller version of itself. Then the code *is* the definition.

K-th symbol in grammar (LC 779). Row 1 is `0`; each row replaces `0 → 01` and `1 → 10`. The k-th symbol of row n is a child of symbol ⌈k/2⌉ of row $n-1$: a left child (odd k) copies its parent, a right child (even k) flips it.

```cpp
int kthGrammar(int n, int k) {
    if (n == 1) return 0;
    int parent = kthGrammar(n - 1, (k + 1) / 2);
    return (k % 2 == 1) ? parent : 1 - parent;
}
```

Building the rows explicitly is $O(2^n)$ memory. The recursion is $O(n)$ time and never builds anything.

Josephus (LC 1823). n people in a circle, every k-th is removed. Let J(n) be the 0-indexed survivor. After the first removal, the circle of $n-1$ people starts k places later, so

```
J(1) = 0
J(n) = (J(n - 1) + k) % n
```

Four lines of code for a problem that is painful to simulate.

4.4 Recursion on linked lists and strings

The "handle the head, trust the rest" idea from Topic 03 is the same pattern:

```cpp
// reverse a linked list recursively
ListNode* reverse(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* newHead = reverse(head->next);   // trust: the rest is reversed
    head->next->next = head;                   // the old second node points back
    head->next = nullptr;
    return newHead;
}

// merge two sorted lists (LC 21)
ListNode* merge(ListNode* a, ListNode* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->val <= b->val) { a->next = merge(a->next, b); return a; }
    b->next = merge(a, b->next); return b;
}
```

Both are $O(n)$ depth. On a list of 10^5 nodes the iterative version is safer — say so.

4.5 Recursion on a stack, with no other container

A GfG favourite. The call stack is your second container.

```cpp
void insertAtBottom(stack<int>& st, int x) {
    if (st.empty()) { st.push(x); return; }
    int top = st.top(); st.pop();      // hold it in this frame
    insertAtBottom(st, x);
    st.push(top);                      // put it back on the way up
}
void reverseStack(stack<int>& st) {
    if (st.empty()) return;
    int top = st.top(); st.pop();
    reverseStack(st);                  // trust: the rest is reversed
    insertAtBottom(st, top);           // the old top belongs at the bottom
}
```

Sorting a stack recursively is the same shape: pop the top, sort the rest, then *insert in sorted position* instead of at the bottom. O(n²).

4.6 Divide and conquer

Divide and conquer is recursion with three steps:

1. **Divide** the input into parts (usually halves).
2. **Conquer** each part recursively.
3. **Combine** the parts' answers.

The **combine** step is where the work and the cleverness live.

Merge sort — learn this one cold

```cpp
void mergeSort(vector<int>& a, int lo, int hi, vector<int>& tmp) {
    if (lo >= hi) return;                          // 0 or 1 element is sorted
    int mid = lo + (hi - lo) / 2;
    mergeSort(a, lo, mid, tmp);                    // trust: left half sorted
    mergeSort(a, mid + 1, hi, tmp);                // trust: right half sorted
    int i = lo, j = mid + 1, k = lo;               // combine: merge two sorted runs
    while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi)  tmp[k++] = a[j++];
    for (k = lo; k <= hi; k++) a[k] = tmp[k];
}
```

O(n log n) always, O(n) extra space, **stable** (the `<=` keeps equal elements in their original order).

Counting while merging

The reason merge sort matters in interviews is that **at the moment of the merge, both halves are sorted** — and that lets you count cross pairs in linear time.

- **Count inversions** (GfG classic): pairs i < j with a[i] > a[j]. During the merge, whenever you take from the right half, every element still waiting in the left half is greater — add `mid - i + 1`.
- **Reverse pairs (LC 493)**: pairs with a[i] > 2·a[j]. The condition differs from the merge order, so count with a separate two-pointer sweep *before* merging.

Quick sort

```
int partition(vector<int>& a, int lo, int hi) {     // Lomuto
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    return i;                                       // pivot's final position
}
void quickSort(vector<int>& a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
```

Here the work is in the **divide** step, and combine is free. O(n log n) on average, **O(n²) on already-sorted input** with a last-element pivot — pick a random pivot to fix it. Not stable. In place.

	MERGE SORT	QUICK SORT
Where the work is	combine (merge)	divide (partition)
Worst case	O(n log n)	O(n²)
Extra space	O(n)	O(log n) stack on average
Stable	yes	no
Used for	linked lists, counting problems, external sorting	in-memory arrays (`std::sort` is introsort)

Divide and conquer on expressions (LC 241)

"Every way to parenthesise `2*3-4*5`": pick each operator as the **last** one evaluated, solve the left and right sides recursively, and combine every left answer with every right answer. Memoise on the substring range because the same sub-expression appears in many splits.

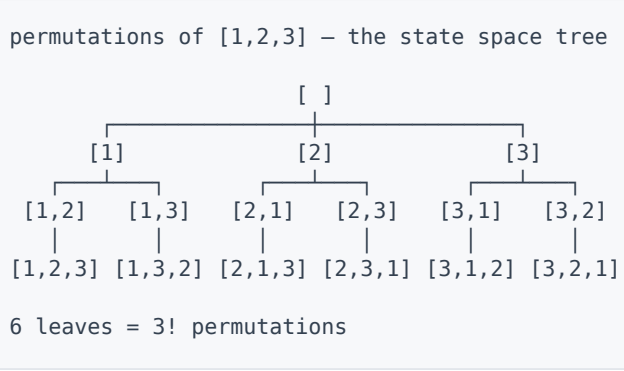
PART 5 — Backtracking: the one template

5.1 The idea

Backtracking builds a solution **one choice at a time**. At each step:

1. **Choose** — make one of the available choices.
2. **Explore** — recurse to make the remaining choices.
3. **Un-choose** — undo the choice, so the next option starts from a clean state.

Picture the **state space tree**: the root is "nothing chosen yet", each edge is one choice, each leaf is a complete candidate. Backtracking is a depth-first walk of that tree. Un-choosing is how you walk back up an edge.



5.2 The template

Memorise this shape. Every problem from Part 6 to Part 10 is this with different blanks.

```
void backtrack(<state>, vector<T>& path, vector<vector<T>>& res) {
    if (<path is a complete answer>) {      // 1. goal test
        res.push_back(path);
        return;
    }
    for (<each choice available now>) {     // 2. iterate the choices
        if (<choice is invalid>) continue;  // 3. prune
        <apply the choice>;                 //    choose
        path.push_back(choice);
        backtrack(<smaller state>, path, res);  // explore
        path.pop_back();                    //    un-choose
        <undo the choice>;
    }
}
```

The five blanks, and the questions that fill them:

BLANK	QUESTION	SUBSETS	PERMUTATIONS	N-QUEENS
state	what tells me where I am?	index `i` / `start`	`used[]`	row `r`
goal	when is a path complete?	every node / `i == n`	`path.size() == n`	`r == n`
choices	what can I pick now?	elements from `start`	any unused element	each column
valid	which choices are illegal?	none	already used	attacked square
undo	what did "choose" change?	`pop_back`	`pop_back`, `used = false`	clear col and diagonals

5.3 The undo must exactly mirror the do

Every line in "apply the choice" needs a matching line in "undo the choice", **in reverse order**. If you set three flags, clear three flags. If you modified the board, restore the board.

Forgetting the undo is bug #1 of this topic. The symptom is distinctive: the **first** answer is correct and every later one is missing elements or contains extras, because later branches inherit garbage from earlier ones.

5.4 Two shapes of the same tree

There are two ways to write subset-style backtracking, and you need both.

Shape A — pick / skip. Each level decides about **one element**. Two branches per level. Answers are at the **leaves**.

```
void dfs(int i) {
    if (i == n) { res.push_back(path); return; }
    path.push_back(a[i]); dfs(i + 1); path.pop_back();   // pick a[i]
    dfs(i + 1);                                          // skip a[i]
}
```

Shape B — loop from start. Each level decides **which element comes next**. A `for` loop over `start..n-1`. Answers are at **every node**.

```
void dfs(int start) {
    res.push_back(path);                          // every node is an answer
    for (int i = start; i < n; i++) {
        path.push_back(a[i]);
        dfs(i + 1);                               // only later elements next
        path.pop_back();
    }
}
```

Both enumerate the same 2^n subsets. Use **A** when every element gets an independent yes/no (subset sums, target sum, the knapsack family that becomes DP). Use **B** when you need to **skip duplicates** or **stop early on a sorted array** — both of those need a loop to `continue` or `break` out of.

5.5 Pruning — the difference between TLE and accepted

The template visits every node of the tree. Pruning cuts subtrees that cannot contain an answer. Four kinds come up again and again:

- **Bound pruning** — on sorted input, once `a[i] > remaining`, every later element is too big as well: `break`, not `continue`. (Combination Sum.)
- **Count pruning** — not enough elements left to finish: `i <= n - (k - path.size())` (Combinations); "the remaining digits cannot fill the remaining IP segments" (Restore IP).
- **Validity pruning** — check constraints when you *place* a piece, not when the board is full. (N-Queens, Sudoku.)
- **Symmetry pruning** — two branches that are mirror images explore identical subtrees; try only one. (Duplicates in Subsets II; equal-sum buckets in Partition to K Equal Sum Subsets.)

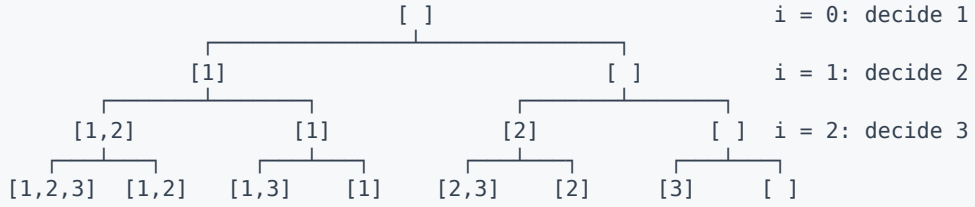
> **Prune early, near the root.** A check that cuts one branch at depth 2 removes far more of the tree than the same check at depth 15. That is why LC 526 fills positions from n downward, and why LC 698 sorts the numbers in decreasing order.

PART 6 — Subsets and subsequences

6.1 All subsets (LC 78) — pick / skip

```
vector<vector<int>> res;
vector<int> path;
void dfs(const vector<int>& a, int i) {
    if (i == (int)a.size()) { res.push_back(path); return; }
    path.push_back(a[i]);
    dfs(a, i + 1);              // pick
    path.pop_back();
    dfs(a, i + 1);              // skip
}
```

The recursion tree for `[1,2,3]`:



2^n leaves, n levels, O(n) to copy each answer: $O(n \cdot 2^n)$ time, O(n) recursion space.

6.2 Subsets with duplicates (LC 90) — sort, then skip at the same depth

With `[1,2,2]`, the plain version produces `[1,2]` twice: once with the first 2 and once with the second. The fix is one line, but you must understand *why* it works.

```
sort(a.begin(), a.end());                       // duplicates become adjacent
void dfs(const vector<int>& a, int start) {
    res.push_back(path);
    for (int i = start; i < (int)a.size(); i++) {
        if (i > start && a[i] == a[i - 1]) continue;   // THE line
        path.push_back(a[i]);
        dfs(a, i + 1);
        path.pop_back();
    }
}
```

Why `i > start` and not `i > 0`? Inside one call, the loop is choosing *which value goes in the next slot*. Choosing the first `2` or the second `2` for the same slot leads to identical subtrees, so only the first is allowed. That is "skip duplicates **at the same depth**". But picking a `2` for this slot and *another* `2` for the next slot is legitimate — that is how you get `[2,2]` — and that happens at a *deeper* call where `i == start`, so the check does not fire.

```
[1,2,2] sorted. Same depth, same value -> skip.

                      [ ]
          ┌──────────────┼──────────────┐
         [1]            [2]          (2) skipped: i > start, a[i] == a[i-1]
       ┌──┴──┐           │
    [1,2]   (2)x      [2,2]
      │
   [1,2,2]
```

This exact line appears in Subsets II, Combination Sum II, 3Sum and 4Sum. Learn it once.

6.3 Subsets with a bitmask

For n ≤ 20, every subset corresponds to an integer 0 … $2^n - 1$: bit j set means "element j is in".

```cpp
for (int mask = 0; mask < (1 << n); mask++) {
    vector<int> s;
    for (int j = 0; j < n; j++)
        if (mask >> j & 1) s.push_back(a[j]);
    res.push_back(s);
}
```

No recursion at all. Useful when you need to iterate over subsets inside some other loop, and it reappears in bitmask DP (Topic 11) and Bit Manipulation (Topic 13).

6.4 Subsequences with a property — return a value instead of building lists

"Count the subsequences whose sum is k" is the same pick / skip tree, but now the function **returns** a count, and nothing is pushed anywhere:

```cpp
int count(const vector<int>& a, int i, int sum, int k) {
    if (i == (int)a.size()) return sum == k ? 1 : 0;
    return count(a, i + 1, sum + a[i], k)      // pick
         + count(a, i + 1, sum, k);            // skip
}
```

Three variants every placement candidate should be able to write from this one function:

ASKED FOR	CHANGE
print all subsequences with sum k	push / pop a `path`, print at the leaf
print only **one**	return `bool`; `if (pick) return true;` stops the search
count them	return `int`, add the two branches (above)

The "print only one" variant is the **early-exit** technique: once a branch returns `true`, the caller returns `true` immediately without exploring the rest. Sudoku uses exactly this.

6.5 When subsets turn into DP

`count(a, i, sum, k)` depends only on `(i, sum)`. If the same `(i, sum)` pair is reached by different paths, the subtree below it is identical — so cache it. That turns the $O(2^n)$ recursion into $O(n \cdot \text{range-of-sums})$. This is **Target Sum (LC 494)**, **Partition Equal Subset Sum (LC 416)**, and the 0/1 knapsack. Part 11 is about that transition.

PART 7 — Combinations and combination sums

7.1 Choose k from n (LC 77)

Shape B with a depth limit and a count prune:

```
void dfs(int n, int k, int start) {
    if ((int)path.size() == k) { res.push_back(path); return; }
    int need = k - (int)path.size();
    for (int i = start; i <= n - need + 1; i++) {   // leave room for `need` numbers
        path.push_back(i);
        dfs(n, k, i + 1);
        path.pop_back();
    }
}
```

Why `start`? It forces every combination to be built in increasing order, so `[1,2]` and `[2,1]` cannot both appear. Combinations are unordered; the `start` index is how you tell the recursion so.

C(n, k) answers, O(k · C(n, k)) time.

7.2 The combination-sum family

Three problems, one skeleton, and the differences are exactly the three things worth memorising.

	REUSE ALLOWED?	DUPLICATES IN INPUT?	NEXT CALL	EXTRA LINE
LC 39 Combination Sum	yes	no	`dfs(i, …)`	–
LC 40 Combination Sum II	no	yes	`dfs(i + 1, …)`	`if (i > start && c[i] == c[i-1]) continue;`
LC 216 Combination Sum III	no (digits 1–9)	no	`dfs(d + 1, …)`	stop when `path.size() == k`

```
// LC 39 — the base version
sort(c.begin(), c.end());
void dfs(const vector<int>& c, int start, int remain) {
    if (remain == 0) { res.push_back(path); return; }
    for (int i = start; i < (int)c.size(); i++) {
        if (c[i] > remain) break;            // sorted: everything after is bigger
        path.push_back(c[i]);
        dfs(c, i, remain - c[i]);            // i: may use c[i] again
        path.pop_back();
    }
}
```

- `i` instead of `i + 1` is what allows reuse. With `start` it still prevents `[3,2,2]` from appearing next to `[2,2,3]`.
- `break` instead of `continue` is the bound prune, and it only works because the array is sorted.

- Passing `remain` **by value** means no undo is needed for it — only `path` is shared.

7.3 Letter combinations of a phone number (LC 17)

The choices change with depth: at level i, the options are the letters of `digits[i]`.

```
void dfs(const string& d, int i) {
    if (i == (int)d.size()) { res.push_back(path); return; }
    for (char ch : keys[d[i] - '0']) {
        path.push_back(ch);
        dfs(d, i + 1);
        path.pop_back();
    }
}
```

At most 4^n answers. Remember that an empty input returns `[]`, not `[""]` — the most common wrong answer on this problem.

PART 8 — Permutations

8.1 Distinct values (LC 46) — the `used` array

Permutations are ordered, so any unused element may go in the next slot — no `start` index.

```
void dfs(const vector<int>& a) {
    if (path.size() == a.size()) { res.push_back(path); return; }
    for (int i = 0; i < (int)a.size(); i++) {
        if (used[i]) continue;
        used[i] = true;  path.push_back(a[i]);
        dfs(a);
        used[i] = false; path.pop_back();     // undo BOTH
    }
}
```

n! leaves, O(n · n!) time, O(n) depth. n ≤ 8 runs instantly; n = 10 is 3.6 million answers; n = 12 is 479 million and will not finish. That is why permutation problems always have tiny constraints.

8.2 The swap method

The other classic: fix position `i` by swapping each candidate into it, recurse on `i + 1`, swap back.

```
void permute(vector<int>& a, int i) {
    if (i == (int)a.size()) { res.push_back(a); return; }
    for (int j = i; j < (int)a.size(); j++) {
        swap(a[i], a[j]);            // choose a[j] for position i
        permute(a, i + 1);
        swap(a[i], a[j]);            // undo
    }
}
```

No `used` array and no `path` — the array itself is the state. It is the version GfG and many textbooks show. It is harder to make work with duplicates, so prefer the `used` array version when the input can repeat.

8.3 Permutations with duplicates (LC 47)

Sort, then add one condition:

```
if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
```

Read it as a rule: **equal values must be placed in their left-to-right order.** If the previous twin is *not* used yet, placing this one now would start an ordering that the previous twin already explored. That single rule removes every duplicate permutation.

(Writing `used[i - 1]` without the `!` also produces correct output, with a less efficient tree. If an interviewer asks, either works; the `!used` version prunes earlier.)

8.4 The k-th permutation directly (LC 60)

You do not need to generate all n! permutations to find the k-th. There are (n − 1)! permutations starting with each digit, so the first digit is `digits[(k − 1) / (n − 1)!]`. Remove it, set `k` to the remainder, recurse on n − 1 digits. O(n²) instead of O(n!).

8.5 Next permutation

`std::next_permutation(a.begin(), a.end())` rearranges into the next lexicographic order and returns `false` after the last one. It is iterative and handles duplicates correctly. The algorithm (LC 31) — find the rightmost ascent, swap with the next larger element to its right, reverse the suffix — is a common standalone interview question.

PART 9 — Constraint backtracking on boards and grids

9.1 N-Queens (LC 51, LC 52)

Place n queens on an n × n board so that no two share a row, column or diagonal.

Key decisions, and they are what the interviewer is grading:

1. **One queen per row** — so the recursion depth *is* the row, and the row constraint is satisfied for free.
2. **O(1) attack checks.** Do not scan the board. Keep three boolean arrays:

```
cells on the same "\" diagonal share r - c      (add n-1 to make it >= 0)
cells on the same "/" diagonal share r + c

       c=0  c=1  c=2  c=3              r+c values        r-c+3 values
r=0     .    .    .    .              0 1 2 3            3 2 1 0
r=1     .    .    .    .              1 2 3 4            4 3 2 1
r=2     .    .    .    .              2 3 4 5            5 4 3 2
r=3     .    .    .    .              3 4 5 6            6 5 4 3
```

```cpp
void place(int r, int n) {
    if (r == n) { res.push_back(board); return; }
    for (int c = 0; c < n; c++) {
        if (col[c] || diag[r + c] || anti[r - c + n - 1]) continue;
        col[c] = diag[r + c] = anti[r - c + n - 1] = true;
        board[r][c] = 'Q';
        place(r + 1, n);
        board[r][c] = '.';
        col[c] = diag[r + c] = anti[r - c + n - 1] = false;
    }
}
```

There are **2n − 1 diagonals** in each direction, so size the arrays `2 * n - 1`. Answers: n = 4 → 2, n = 8 → 92. Time is bounded by O(n!) and is far smaller in practice because of pruning.

The bitmask version (LC 52). Pass `cols`, `d1`, `d2` as integers **by value**. Then each call has its own copy, and there is nothing to undo — a clean demonstration of Part 1.4.

9.2 Sudoku solver (LC 37)

Same structure, with three changes:

- The **state** is the index into a list of blank cells.
- The **choices** are the digits 1–9.
- It is a **search for one answer**, so the function returns `bool` and stops as soon as one branch succeeds.

```cpp
bool solve(vector<vector<char>>& b, int idx) {
    if (idx == (int)blanks.size()) return true;
    auto [r, c] = blanks[idx];
    int bx = (r / 3) * 3 + c / 3;                  // which 3x3 box
    for (int d = 1; d <= 9; d++) {
        if (row[r][d] || col[c][d] || box[bx][d]) continue;
        row[r][d] = col[c][d] = box[bx][d] = true;  b[r][c] = '0' + d;
        if (solve(b, idx + 1)) return true;         // found it: stop everything
        row[r][d] = col[c][d] = box[bx][d] = false; b[r][c] = '.';
    }
    return false;                                   // nothing fits: go back
}
```

`(r / 3) * 3 + c / 3` numbers the boxes 0–8 left to right, top to bottom. Memorise it; it is also the key to LC 36 Valid Sudoku.

Do not undo after a success. Once `solve` returns `true` the board must keep its digits — that is why the `return true` comes *before* the undo lines.

9.3 Word search (LC 79) — mark, explore, unmark

Grid backtracking: from each cell, try the four neighbours. A cell may be used once **per path**, so mark it while it is on the path and unmark it on the way back.

```cpp
bool dfs(vector<vector<char>>& b, const string& w, int i, int r, int c) {
    if (i == (int)w.size()) return true;
    if (r < 0 || c < 0 || r >= R || c >= C || b[r][c] != w[i]) return false;
    char saved = b[r][c];
    b[r][c] = '#';                                  // on the current path
    bool found = dfs(b, w, i + 1, r + 1, c) || dfs(b, w, i + 1, r - 1, c)
              || dfs(b, w, i + 1, r, c + 1) || dfs(b, w, i + 1, r, c - 1);
    b[r][c] = saved;                                // off the path again
    return found;
}
```

Backtracking DFS versus graph DFS. In graph DFS (Topic 12) a visited cell stays visited forever — you only want to reach each cell once. In backtracking DFS you unmark, because a cell blocked for *this path* must be available to *other paths*. Mixing the two up is a classic bug: never unmarking makes word search miss answers; unmarking in a flood fill makes it exponential.

Cost: $O(R \cdot C \cdot 4 \cdot 3^{L-1})$ for a word of length L — four directions at the first step, then at most three (you cannot go back to the cell you came from).

9.4 The rest of the grid family

- **Rat in a maze** (GfG classic) — find every path from (0, 0) to (n − 1, n − 1) moving D/L/R/U through open cells. Exactly word search with a path string instead of a word, and results collected at the target cell. Try directions in alphabetical order (`D, L, R, U`) if the output must be sorted.
- **Unique Paths III (LC 980)** — count paths that visit **every** empty cell exactly once. Mark, recurse, unmark; at the end cell, accept only if the remaining-cell counter is zero.
- **Word Search II (LC 212)** — many words on one board. Running LC 79 once per word is too slow; build a **trie** of all the words and walk the board and the trie together, so every word sharing a prefix shares the search.

- **M-colouring** (GfG) — colour the vertices of a graph with at most m colours so no edge joins two equal colours. Vertex by vertex, try each colour, check neighbours, undo.

PART 10 — Partitioning and constrained generation

10.1 Cut at every position

A large family asks you to split a string into pieces, each satisfying a rule. The choice at each step is **where the next piece ends**.

```cpp
void dfs(const string& s, int start) {
    if (start == (int)s.size()) { res.push_back(path); return; }
    for (int end = start; end < (int)s.size(); end++) {
        string piece = s.substr(start, end - start + 1);
        if (!valid(piece)) continue;
        path.push_back(piece);
        dfs(s, end + 1);
        path.pop_back();
    }
}
```

PROBLEM	VALID(PIECE)	EXTRA
LC 131 Palindrome Partitioning	piece is a palindrome	precompute `isPal[i][j]` with DP for speed
LC 93 Restore IP Addresses	1–3 digits, ≤ 255, no leading zero	exactly 4 pieces; prune on remaining length
LC 140 Word Break II	piece is in the dictionary	memoise on `start`
LC 282 Expression Add Operators	any number without a leading zero	also choose `+`, `-` or `*` before each piece

There are n − 1 gaps in a string of length n and each may or may not be cut, so the tree has up to $\mathbf{2^{n-1}}$ leaves.

10.2 Generation with counters (LC 22)

Generate Parentheses is the cleanest example of **validity pruning**. Instead of generating all 2^{2n} strings and filtering, only ever take a branch that can still lead to a valid string:

- add `(` only while `open < n`;
- add `)` only while `close < open`.

Every leaf is then valid by construction. The number of answers is the **Catalan number** C(n) — 1, 1, 2, 5, 14, 42 … — which grows like 4^n / n^1.5.

10.3 Remove Invalid Parentheses (LC 301)

First **count** the minimum deletions: scan once, and every `)` with no open partner must go, as must every `(` left over at the end. Then backtrack with those two budgets: at each bracket, either delete it

(if budget remains) or keep it (if the running balance stays non-negative). Collecting into a set removes duplicates caused by deleting either of two adjacent identical brackets.

10.4 Expression Add Operators (LC 282) — the `last` trick

Between digits you may place `+`, `-` or `*`. The difficulty is `*`, which binds tighter. Carry two numbers down the recursion: `val`, the value so far, and `last`, the last term added. When you multiply by `x`, undo the last term and add it back multiplied:

```
val_new  = val - last + last * x
last_new = last * x
```

So `1 + 2 * 3`: after `1 + 2`, val = 3 and last = 2; multiplying by 3 gives 3 − 2 + 6 = 7. Use `long long` — intermediate products overflow `int`.

PART 11 — From recursion to memoization: the bridge to DP

11.1 Overlapping subproblems

Look again at the Fibonacci tree in Part 1.5: `fib(3)` is computed twice, `fib(2)` three times. For `fib(40)` the naive recursion makes over 300 million calls to compute 41 distinct values.

If a function's result depends **only on its parameters** (no global state it modifies), then the same parameters always give the same answer. Store it the first time; return it every time after.

```
int memo[100];                                  // fill with -1 first
int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];          // seen before: O(1)
    return memo[n] = fib(n - 1) + fib(n - 2);  // compute once, store
}
```

$O(2^n)$ → **O(n)**. That is memoization, also called **top-down dynamic programming**.

11.2 When does memoization help?

Ask two questions:

1. **Does the function return a value** that depends only on its parameters? Enumeration problems (print all subsets) do not — every leaf is a different answer and there is nothing to share.
2. **Can the same parameters be reached by different paths?** If the state is `(i, sum)` and many different choice sequences reach the same `(i, sum)`, yes.

PROBLEM	STATE	REPEATS?	MEMO HELPS?
All subsets (LC 78)	`(i, path)`	no — `path` differs	no
Count subsets with sum k	`(i, sum)`	yes	**yes**
Target Sum (LC 494)	`(i, sum)`	yes	**yes**
Word Break II (LC 140)	`start`	yes	**yes** — even though it returns lists
Different Ways to Add Parentheses (LC 241)	`(lo, hi)`	yes	**yes**
N-Queens	`(row, cols, diagonals)`	rarely	no

11.3 The recipe

1. Write the plain recursion first and make it correct.
2. Identify the **parameters that change** between calls — that is the state.

3. Make a table indexed by the state, filled with a sentinel (`-1`).
4. At the top of the function: if the table has an answer, return it.
5. At every `return`: store before returning.

Bottom-up DP (Topic 11) is the same table filled with loops instead of recursion. Every DP problem you will meet can be reached by this route — **recursion first, memo second, table third** — and that is the order to use under interview pressure.

> **Negative indices.** Target Sum's running sum can be negative. Shift it: store at `memo[i][sum + total]` where `total` is the sum of all elements. Forgetting the shift is an out-of-bounds write.

PART 12 — The complete pattern taxonomy

Every problem in Part 16 is labelled with one of these twelve. When you can name the pattern, you already have most of the code.

#	PATTERN	RECOGNISE IT BY	CORE TOOL
1	Reduce by one	"handle one element, the rest is the same problem"	base case + one call on n − 1
2	Halve the input	power, binary search, grammar rows	one call on n / 2, **stored**
3	Divide and conquer with a combine	sort, count pairs across halves, split an expression	two calls + a linear merge
4	Recurrence from the definition	Josephus, Hanoi, "row n is built from row n − 1"	write the formula as code
5	Pick / skip	subsets, subsequences, "each element in or out"	two calls per level, answer at leaves
6	Loop from `start`	combinations, sums to target, unordered picks	`for (i = start…)` + `dfs(i + 1)`
7	Permutations	arrangements, orderings, "every order"	`used[]` array, or swap into place
8	Duplicate skipping	input has repeated values, "no duplicate answers"	sort + `i > start && a[i] == a[i-1]`
9	Partition at every cut	split a string into valid pieces	`for (end = start…)` + `dfs(end + 1)`
10	Generation with validity counters	parentheses, case flips, IPs	only branch where the rule still holds
11	Constraint placement	queens, Sudoku, buckets, colouring	O(1) conflict arrays, set / clear
12	Grid path backtracking	word search, maze paths, visit every cell	mark → 4 directions → unmark

And one modifier that applies to several of them:

+ Memo — when a counting or optimising recursion revisits the same state (patterns 4, 5, 9), add a cache keyed on that state (Part 11).

How to tell 5 and 6 apart

They generate the same subsets. The difference is **what each level of the recursion decides**:

- **Pattern 5** — each level decides **one element: in or out.** Two branches, always. Natural when the answer is a count or a yes/no, and it is the shape that becomes knapsack DP.

- **Pattern 6** — each level decides **which element comes next.** A loop over the remaining elements. Required when you need to `continue` past a duplicate or `break` on a sorted bound.

If you need duplicate skipping or a sorted-bound break, use 6. If you are going to memoise, use 5.

How to tell 6 and 7 apart

- Is `[1,2]` the same answer as `[2,1]`? **Yes** → combinations → pattern 6, with a `start` index.
- **No** → permutations → pattern 7, with a `used` array and the loop starting from 0.

PART 13 — How to approach ANY recursion or backtracking problem

Six steps. They take two minutes and they prevent the three expensive mistakes: the wrong recursion shape, a missing undo, and an exponential solution where the constraints did not allow one.

Step 1 — Read the constraints first

N IS ABOUT	AFFORDABLE	MEANS
≤ 10	O(n!)	permutations are fine
≤ 20	$O(2^n \cdot n)$	subsets / pick-skip are fine
≤ 30 with pruning	$O(2^n)$ with heavy pruning or meet-in-the-middle	careful
≥ 100	polynomial only	recursion **must** be memoised, or it is DP / greedy

If n is large and you are writing exponential backtracking, stop — the intended solution is something else.

Step 2 — Decide: enumerate, count, or search?

- **Enumerate** ("return all …") → `void` function, push into `res`, path by reference + undo.
- **Count / optimise** ("how many", "maximum") → function **returns** the number; consider memo.
- **Search** ("is there one", "solve the board") → function returns `bool`; `if (dfs(...)) return true;` stops early.

Step 3 — Draw the top two levels of the tree

What is at the root? What are the edges out of the root — one per element (pattern 6/7), two per element (pattern 5), one per cut (pattern 9), one per cell or digit (pattern 11)? Thirty seconds on paper here saves ten minutes of rewriting.

Step 4 — Fill the five blanks

From Part 5.2: **state, goal, choices, validity check, undo.** Say each one out loud. If you cannot say what the undo is, you have not decided what the choice changes.

Step 5 — Add the pruning

- Sorted input with a target? `break` on the bound.
- Duplicates in the input? Sort, and skip at the same depth.
- Constraint problem? Check validity when **placing**, with O(1) arrays.

- Can a count tell you the branch is hopeless? (Not enough elements left; remaining length too short or too long.)

Step 6 — Dry-run the smallest inputs

1. **Empty input** — `[]`, `""`, n = 0. What should come back: `[]` or `[[]]`? (Subsets of `[]` is `[[]]`; letter combinations of `""` is `[]`.)
2. **One element.**
3. **All elements equal** — `[2,2,2]`. This is the input that catches a missing or wrong duplicate skip.
4. **No answer exists** — target unreachable, n = 2 or 3 for N-Queens.
5. **The given example**, checked against the expected count, not just the first answer.

PART 14 — The bug catalogue

Fourteen bugs account for nearly every wrong answer in this topic. When a submission fails, read this list before re-reading your code.

1. Forgetting to undo the choice. `path.push_back(x)` without the matching `pop_back()`, `used[i] = true` without the reset. Symptom: the first answer is right; later answers carry stale elements, or answers go missing. Every "do" line needs a mirror "undo" line, in reverse order.

2. Pushing a reference that keeps changing. In C++, `res.push_back(path)` copies, which is right. The bug appears when you store a pointer, or in Java/Python ports where `res.add(path)` stores the same list object. Symptom: every answer in `res` is identical (usually empty).

3. Missing or wrong base case. No base case, or one the recursion can step over (`n == 0` when n can jump from 1 to −1). Symptom: stack overflow, segmentation fault, or exit code `0xC00000FD` on Windows.

4. Not making progress. `dfs(i, …)` where you meant `dfs(i + 1, …)` in a no-reuse problem; `f(n)` calling `f(n)`. Symptom: infinite recursion. In Combination Sum, `i` is correct (reuse); in Combination Sum II it is the bug.

5. `i > 0` instead of `i > start` in the duplicate skip. Symptom: answers that legitimately use a repeated value, like `[1,2,2]` or `[2,2]`, disappear.

6. Skipping duplicates without sorting first. The `a[i] == a[i-1]` check only works when equal values are adjacent. Symptom: duplicate answers still appear.

7. `continue` where `break` was intended (or the reverse). On a sorted array, once `c[i] > remain` nothing later fits — `break`. `continue` gives the right answer slowly (TLE); `break` on an *unsorted* array gives a wrong answer.

8. Using `start` for permutations, or not using it for combinations. Symptom: permutations come out incomplete (only increasing orders); combinations come out with every ordering duplicated.

9. Undoing after a successful search. In Sudoku and other "find one" problems, `if (solve()) return true;` must come **before** the undo lines. Symptom: the solver reports success but the board is back to its original state.

10. Passing large state by value without realising it. `void dfs(vector<int> path, string s)` copies both at every call. Symptom: correct answers, TLE or MLE on the large tests. Pass by reference and undo.

11. Calling the same recursion twice instead of storing it. `power(x, n/2) * power(x, n/2)`. Symptom: correct, and O(n) instead of O(log n).

12. Integer overflow. `-INT_MIN` in `myPow`; `2 * a[j]` in Reverse Pairs; products in Expression Add Operators. Symptom: wrong answers only on extreme inputs. Use `long long` before the operation, not after.

13. Graph-DFS marking in a backtracking search (or the reverse). Never unmarking a cell in word search loses paths; unmarking in flood fill makes it exponential. Ask: *must a cell be usable by a different path?* If yes, unmark.

14. Wrong empty-input answer. Subsets of `[]` is `[[]]` (one subset, the empty one). Letter combinations of `""` is `[]`. Generate parentheses with n = 0 is `[""]`. Symptom: fails exactly one test case.

Two more worth naming:

15. Memoising a function that depends on mutable global state. If the answer depends on a `used` array or a board that the parameters do not capture, the cache returns answers for the wrong situation. Symptom: wrong answers that change with the order of exploration.

16. Off-by-one in N-Queens diagonal arrays. There are `2n - 1` diagonals, and `r - c` needs `+ n - 1` to be non-negative. Symptom: out-of-bounds writes that crash only for larger n.

PART 15 — Complexity cheat sheet

ALGORITHM	TIME	SPACE (STACK)	NOTE
Factorial, sum of digits, reverse	O(n)	O(n)	O($\log_{10}$ n) depth for digits
Binary search, fast power	O(log n)	O(log n)	store the half-result
Naive Fibonacci	O(φ^n) ≈ O(1.618^n)	O(n)	O(n) time with memo
Tower of Hanoi	O(2^n)	O(n)	exactly $2^n - 1$ moves
Merge sort	O(n log n)	O(n) buffer + O(log n) stack	stable
Quick sort	O(n log n) avg, O(n^2) worst	O(log n) avg	random pivot
Count inversions / reverse pairs	O(n log n)	O(n)	count during the merge
Subsets / pick-skip	O(n · 2^n)	O(n)	2^n answers
Combinations C(n, k)	O(k · C(n, k))	O(k)	
Combination Sum	O(n^(T/m)) loose	O(T/m)	T = target, m = smallest candidate
Permutations	O(n · n!)	O(n)	n! answers
Letter combinations	O(n · 4^n)	O(n)	
Generate parentheses	O(4^n / √n)	O(n)	Catalan many answers
Palindrome partitioning	O(n · 2^n)	O(n)	O(n^2) precompute of `isPal`
N-Queens	O(n!) bound, much less in practice	O(n)	92 answers for n = 8
Sudoku	O(9^blanks) bound, tiny in practice	O(81)	fixed size — "O(1)"
Word search	O(R · C · 3^L)	O(L)	L = word length
Memoised recursion	O(states × work per state)	O(states)	the DP bound

The sentences to say in interviews:

> *"There are 2^n subsets and copying each one costs O(n), so the total is $O(n \cdot 2^n)$. The recursion depth is n, so extra space is O(n) apart from the output."*
>
> *"Backtracking explores the state space tree depth-first and undoes each choice on the way back, so it only ever holds one path in memory."*
>
> *"The recursion revisits the same (index, sum) pairs, so I'll memoise on that pair — that brings it from $O(2^n)$ to $O(n \cdot S)$."*

PART 16 — The problem list

Work these in order. Every one is solved in the companion **Recursion & Backtracking — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have genuinely struggled with it.

Tier 1 — Learn the mechanics (do all thirteen first)

#	PROBLEM	PATTERN
509	Fibonacci Number	4 — recurrence, + memo
344	Reverse String	1 — reduce by one (both ends)
50	Pow(x, n)	2 — halve the input ⭐
21	Merge Two Sorted Lists	1 — reduce by one, on a list
779	K-th Symbol in Grammar	4 — recurrence from the definition
1823	Find the Winner of the Circular Game	4 — Josephus
78	Subsets	5 — pick / skip ⭐
90	Subsets II	8 — duplicate skipping
46	Permutations	7 — `used` array ⭐
77	Combinations	6 — loop from `start`
39	Combination Sum	6 — reuse allowed ⭐
17	Letter Combinations of a Phone Number	6 — choices change with depth
22	Generate Parentheses	10 — validity counters ⭐

Thirteen problems. When you can write all thirteen from a blank page — especially 78, 46, 39 and 22 — you own the template. Do not move on before that is true.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
40	Combination Sum II	8 — duplicates, no reuse ⭐
216	Combination Sum III	6 — fixed size
47	Permutations II	8 — `!used[i-1]` rule ⭐
131	Palindrome Partitioning	9 — cut at every position ⭐
79	Word Search	12 — mark / unmark ⭐
51	N-Queens	11 — constraint placement ⭐
52	N-Queens II	11 — bitmasks by value
37	Sudoku Solver	11 — search, stop at first ⭐
93	Restore IP Addresses	9 — cut + count pruning
784	Letter Case Permutation	10 — branch only on letters
912	Sort an Array (merge sort)	3 — divide and conquer ⭐
60	Permutation Sequence	7 — skip whole blocks
241	Different Ways to Add Parentheses	3 — split on the last operator, + memo
494	Target Sum	5 — pick sign, + memo
1239	Maximum Length of a Concatenated String with Unique Characters	5 — pick / skip with bitmasks

⭐ **51 N-Queens, 37 Sudoku, 79 Word Search, 46/47 Permutations, 39/40 Combination Sum and 131 Palindrome Partitioning are the most-asked backtracking questions in on-campus drives** — Amazon, Microsoft, Adobe, Samsung, Flipkart, TCS Digital, Infosys. Merge sort (912) is asked as "write it" in almost every written round.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
212	Word Search II	12 — grid DFS walking a trie
140	Word Break II	9 — cut, + memo on `start`
698	Partition to K Equal Sum Subsets	11 — buckets + symmetry pruning
301	Remove Invalid Parentheses	10 — deletion budgets
282	Expression Add Operators	9 — cut + the `last` trick
1255	Maximum Score Words Formed by Letters	5 — pick / skip with a resource
980	Unique Paths III	12 — visit every cell
1079	Letter Tile Possibilities	7 — branch on distinct counts
526	Beautiful Arrangement	7 — fill from the hardest position
493	Reverse Pairs	3 — count during the merge

The classic Indian-placement questions (GfG / written-round favourites)

These appear in written rounds and technical interviews, usually in GfG's wording:

- Print 1 to n and n to 1 **without loops**; sum of first n numbers; factorial; count digits
- Sum of digits; reverse a number; check whether a string is a palindrome — recursively
- **"What does this print?"** on functions with work before and after the call (Part 2.2)
- **Tower of Hanoi** — print the moves, and state that there are $2^n - 1$
- Power of a number in O(log n); GCD by Euclid's algorithm
- Recursive binary search; check whether an array is sorted
- **Reverse a stack** and **sort a stack** using recursion only
- Print all **subsequences** of a string / array; all subsequences with sum k; count them; print only the first
- Print all **permutations** of a string — with and without duplicates
- **N-Queens**; print one solution, and count all
- **Rat in a Maze** — all paths, in lexicographic order (`D L R U`)
- **Sudoku solver**
- **M-colouring** of a graph
- **Merge sort** and **quick sort** — write both; explain stability and the worst case
- **Count inversions** in an array (merge-sort based)
- Josephus problem
- Print all **palindromic partitions** of a string
- Word break — print all ways
- Subset sum — does a subset with sum k exist? (leads into DP)

- Generate all binary strings of length n with no two consecutive 1s
- Phone keypad combinations; all balanced parenthesis strings of n pairs

> **Rat in a Maze** and **N-Queens** are close to guaranteed in some form. In both, the interviewer is checking the same three things: that you **mark before recursing**, that you **unmark after**, and that your **validity check is O(1)**. Say all three out loud.

PART 17 — A three-week plan

Assumes about 1.5 hours a day. Recursion is a small topic in code and a large one in thinking — the third week matters most.

Week 1 — Recursion, without backtracking. Days 1–2: Parts 0–3. Write every function in Part 4.1 from memory, then the "what does this print?" functions in Part 2.2 *on paper* before running them. Practice file TODOs 1–4. Days 3–5: Tier 1's first six — 509, 344, 50, 21, 779, 1823. Days 6–7: merge sort (912) and quick sort, written twice each from a blank page; then count inversions.

Week 2 — The template. Day 1: Part 5, and TODOs 5–9. Then one family per day: subsets (78, 90) → permutations (46, 47) → combinations (77, 39, 40, 216) → generation (17, 22, 784) → partitioning (131, 93). Before writing each one, say the five blanks out loud. After each one, write down which duplicate-skip or prune line it needed and why.

Week 3 — Constraints, grids and memo. N-Queens (51, 52) and TODO 10 on day 1; Sudoku (37) on day 2; Word Search (79) and Rat in a Maze on day 3. Then the memo bridge: 494, 241, 140 — in each, write the plain recursion first, then add the cache. Then Tier 3 as far as you get: 698, 301 and 212 first. Finish by re-solving **46, 39, 51, 79 and 131** on a timer — 15 minutes each, on paper, no compiler.

Ongoing: every time a problem defeats you, log which of the 12 patterns it was and which of the bugs you hit. After thirty problems the log tells you exactly what to drill.

Where this topic leads: Trees (Topic 06) is recursion on a shape you already know — re-read its Part 1 after this one and it will feel easy. Graph DFS (Topic 12) is backtracking without the unmark. Every problem in Dynamic Programming (Topic 11) starts as a recursion from Part 11 of this guide. Tries (Topic 14) are what make Word Search II fast. Nothing here is wasted.

Quick reference card

```cpp
// ===== the three questions =====
// 1. smallest input + its answer         -> base case
// 2. trust the smaller call; use it      -> recursive case
// 3. every call moves toward the base    -> termination

// ===== the backtracking template =====
void dfs(<state>) {
    if (<complete>) { res.push_back(path); return; }
    for (<each choice>) {
        if (<invalid>) continue;
        <do>;   path.push_back(x);
        dfs(<next state>);
        path.pop_back();   <undo>;        // mirror image of <do>
    }
}

// ===== pick / skip (subsets, answers at leaves) =====
if (i == n) { res.push_back(path); return; }
path.push_back(a[i]); dfs(i + 1); path.pop_back();   // pick
dfs(i + 1);                                          // skip

// ===== loop from start (combinations, answers at every node) =====
res.push_back(path);
for (int i = start; i < n; i++) {
    if (i > start && a[i] == a[i - 1]) continue;     // duplicates (after sort)
    if (a[i] > remain) break;                        // sorted bound
    path.push_back(a[i]);
    dfs(i + 1 /* or i if reuse is allowed */, remain - a[i]);
    path.pop_back();
}

// ===== permutations =====
for (int i = 0; i < n; i++) {
    if (used[i]) continue;
    if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;   // duplicates
    used[i] = true;  path.push_back(a[i]);
    dfs();
    used[i] = false; path.pop_back();
}

// ===== N-Queens O(1) checks =====
if (col[c] || diag[r + c] || anti[r - c + n - 1]) continue;   // arrays of 2n-1

// ===== Sudoku box index =====
int box = (r / 3) * 3 + c / 3;

// ===== search for ONE answer: stop early, do NOT undo on success =====
if (solve(idx + 1)) return true;

// ===== grid backtracking =====
char saved = b[r][c];  b[r][c] = '#';     // mark
bool ok = dfs(r+1,c) || dfs(r-1,c) || dfs(r,c+1) || dfs(r,c-1);
b[r][c] = saved;                          // unmark

// ===== fast power: ONE call, stored =====
double half = power(x, n / 2);
return n % 2 ? half * half * x : half * half;

// ===== merge sort =====
if (lo >= hi) return;
int mid = lo + (hi - lo) / 2;
mergeSort(lo, mid); mergeSort(mid + 1, hi);
merge(lo, mid, hi);                       // <= keeps it stable

// ===== memo =====
if (memo[i][s] != -1) return memo[i][s];
return memo[i][s] = <recurrence>;
```

The five rules, one last time:

1. **Define what the function returns, then trust it** for smaller inputs. Do not trace.
2. Answer the three questions before writing code: **base case, recursive case, progress.**
3. **Every "do" has an "undo"**, in reverse order. State passed by value needs no undo.
4. **Read the constraints first:** $n \le 10 \to n!$, $n \le 20 \to 2^n$, larger → memo or something else.
5. **Prune near the root:** sort, skip duplicates at the same depth, `break` on the bound, check validity when placing.